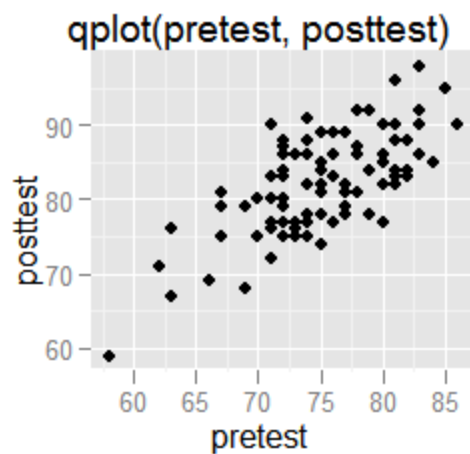
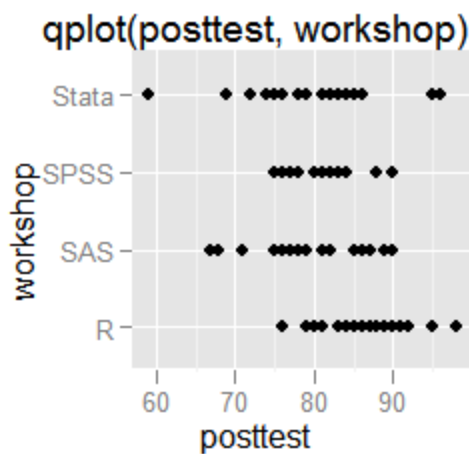
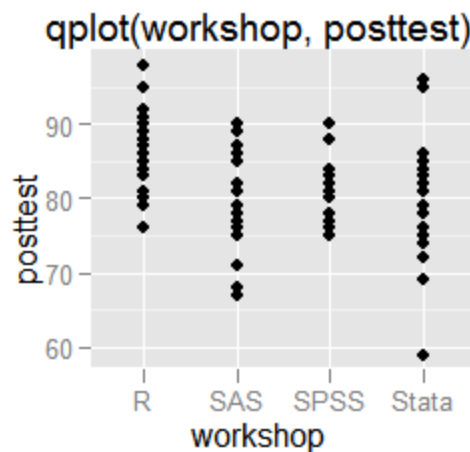
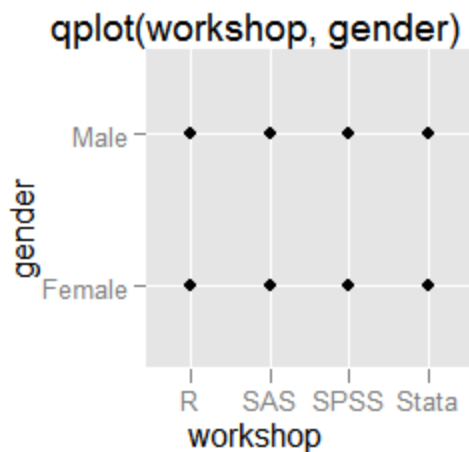
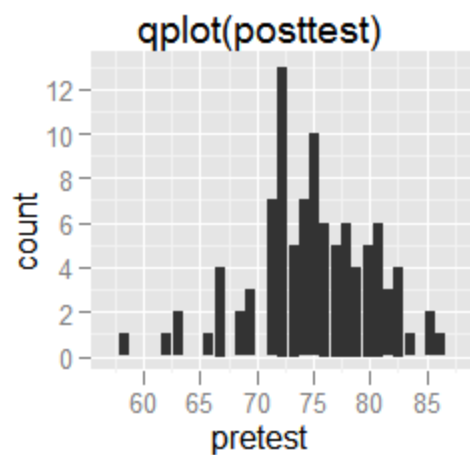
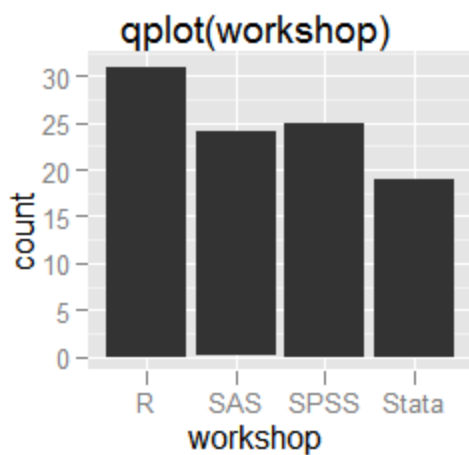


GGPLOT2

The “gg” in ggplot2 stands for the Grammar of Graphics, a comprehensive theory of graphics by **Leland Wilkinson** which he described in his book by the same name. In his book, ***The Grammar of Graphics***, Wilkinson showed how you could describe plots not as discrete types like bar plot or pie chart, but using a “grammar” that would work not only for plots we commonly use but for almost any conceivable graphic.

The ggplot2 package is a simplified implementation of grammar of graphics written by **Hadley Wickham** for R. It is simplified only in that he uses R for data transformation and restructuring, rather than implementing that in his syntax. Wickham’s book, *ggplot2: Elegant Graphics for Data Analysis*, provides a detailed presentation of the ggplot2 package.

To make it easy to get started, the ggplot2 package offers two main functions: `quickplot()` and `ggplot()`. The `quickplot()` function – also known as `qplot()` – mimics R’s traditional `plot()` function in many ways. It is particularly easy to use for simple plots. Below is an example of the default plots that `qplot()` makes. The command that created each plot is shown in the title of each graph. Most of them are useful except for middle one in the left column of `qplot(workshop, gender)`. A plot like that of two factors simply shows the combinations of the factors that exist which is certainly not worth doing a graph to discover.



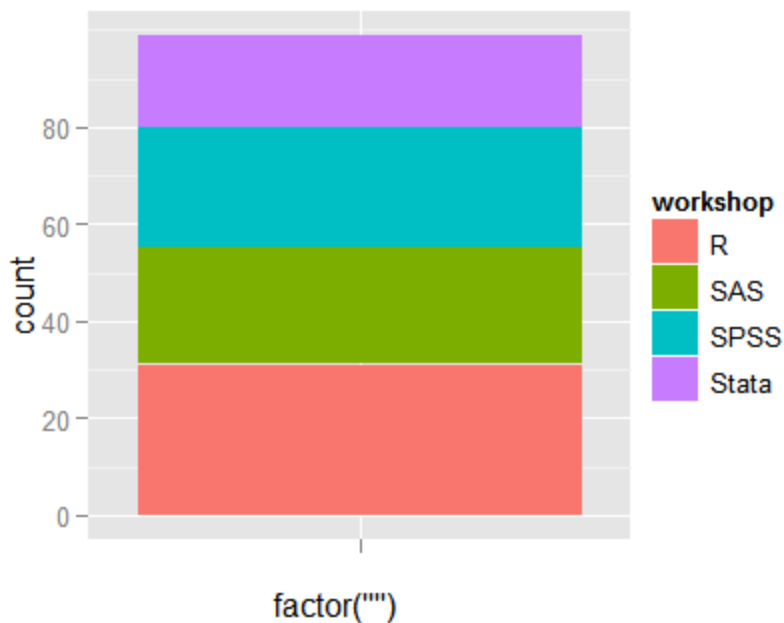
While `qplot()` is easy to use for simple graphs, it does not use the powerful grammar of graphics. The `ggplot()` function does that. To understand `ggplot`, you need to ask yourself, what are the fundamental parts of every data graph? They are:

- **Aesthetics** – these are the roles that the variables play in each graph. A variable may control where points appear, the color or shape of a point, the height of a bar and so on.
- **Geom** – these are the geometric objects. Do you need bars, points, lines?
- **Statistics** – these are the functions like linear regression you might need to draw a line.
- **Scales** – these are legends that show things like circular symbols represent females while circles represent males.
- **Facets** – these are the groups in your data. Faceting by gender would cause the graph to repeat for the two genders.

Let us start our use of the `ggplot()` function with a single stacked bar plot. It is not a very popular plot, but it helps demonstrate how different the grammar of graphics perspective is. On the x-axis there really is no variable, so I plugged in a call to the `factor()` function that creates an empty one on the fly. I then fill the single bar in using the `fill` argument. There is only one type of geometric object on the plot, which I add with `geom_bar`.

Syntax:-

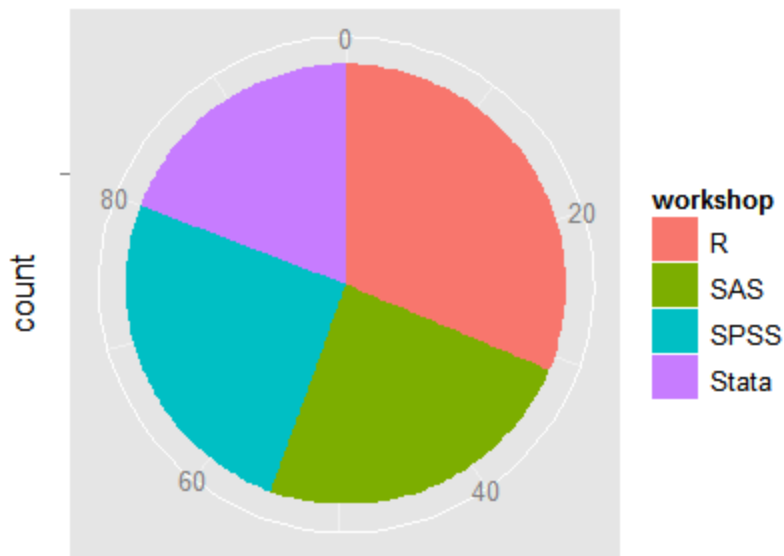
```
ggplot(mydata100, aes(x = factor("")), fill = workshop) +  
  geom_bar()
```



The x-axis comes out labeled as “factor(\\\"\\\")” but we can over-write that with a title for the x-axis. What is particularly interesting is that this can become a pie chart simply by changing its coordinate system to polar. The final line of code changes the label on the discrete x-axis to blank with “\\\"\\”.

Syntax:-

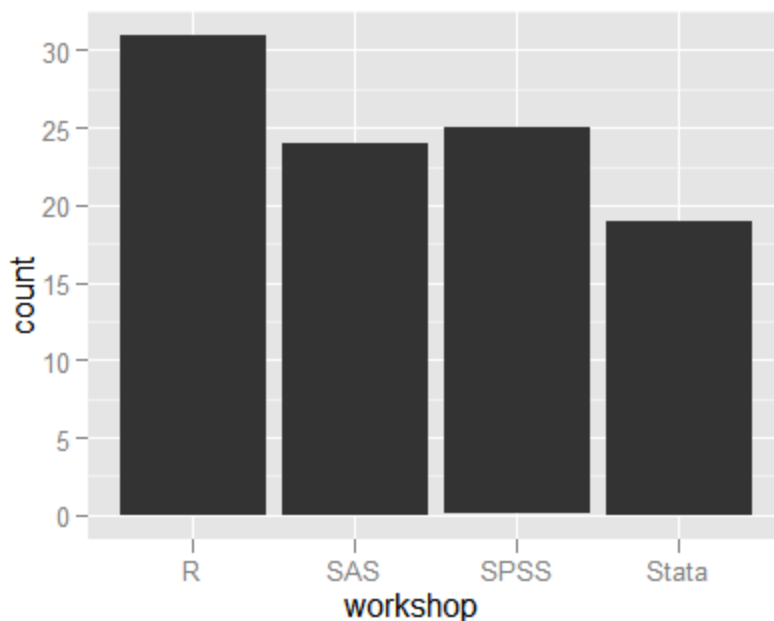
```
ggplot(mydata100, aes(x = factor(\\\"\\\"), fill = workshop) ) + geom_bar() +  
coord_polar(theta = \\\"y\\\") + scale_x_discrete(\\\"\\\")
```



Bar Plots

The upper left corner of the plot of the first plot above shows a bar plot of workshop created with `qplot()`. From the grammar of graphics approach, that graph has only one type of geometric object: bars. The `ggplot()` function itself only needs to specify the data set to use. Note the unusual use of the plus sign “+” to add the effect of `geom_bar()` to `ggplot()`. Only one variable plays an “aesthetic” role: `workshop`. The `aes()` function sets that role. So here is one way to write the code:

```
ggplot(mydata100) + geom_bar( aes(workshop) )
```

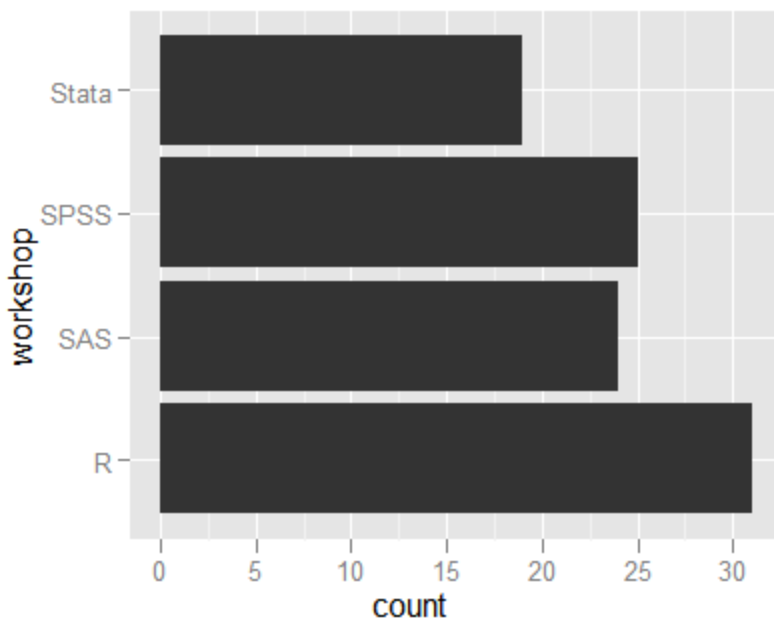


A very useful feature of the `ggplot()` function is that it can pass aesthetic roles to all the functions that are “added” to it. For example, we can create exactly the same barplot with this code:

```
ggplot(mydata100, aes(workshop) ) +  
  geom_bar()
```

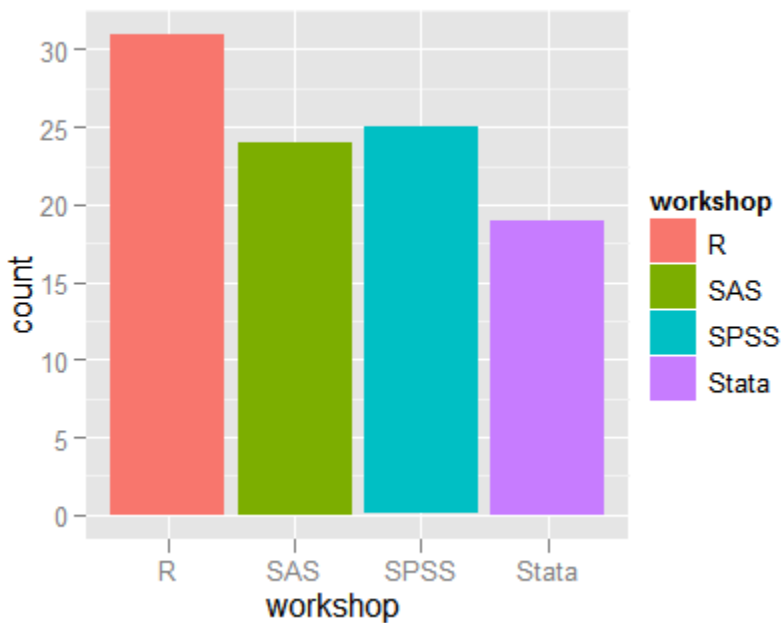
Flipping from vertical to horizontal bars is easy with the addition of the `coord_flip()` function.

```
ggplot(mydata100, aes(workshop) ) +  
  geom_bar() + coord_flip()
```



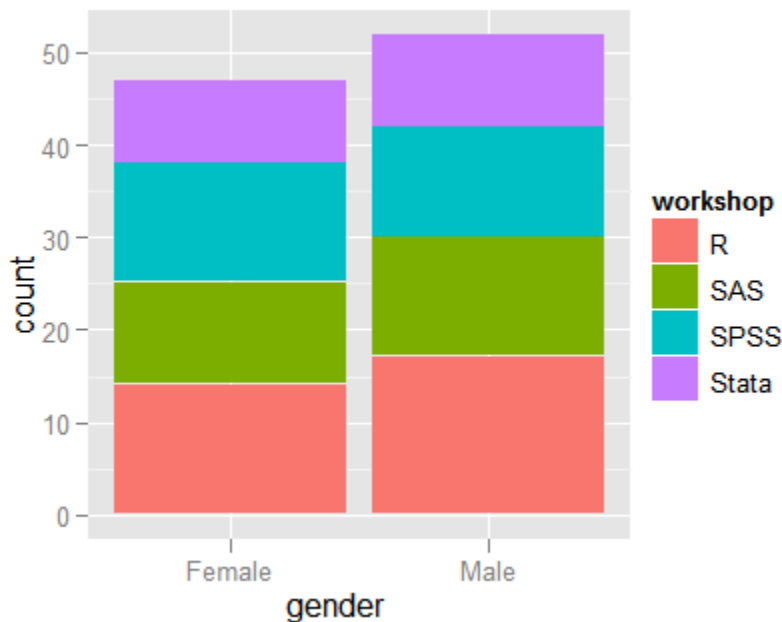
If you want to fill the bars with color, you can do that using the “fill” argument.

```
ggplot(mydata100, aes(workshop, fill =  
workshop ) ) + geom_bar()
```



The use of color above was, well, colorful, but it did not add any useful information. However, when displaying bar plots of two factors, the fill argument becomes very useful. You can display it several ways. Below I use fill to color the bars by workshop and set the “position” to stack.

```
ggplot(mydata100, aes(gender, fill = workshop) )  
+ geom_bar(position = "stack")
```



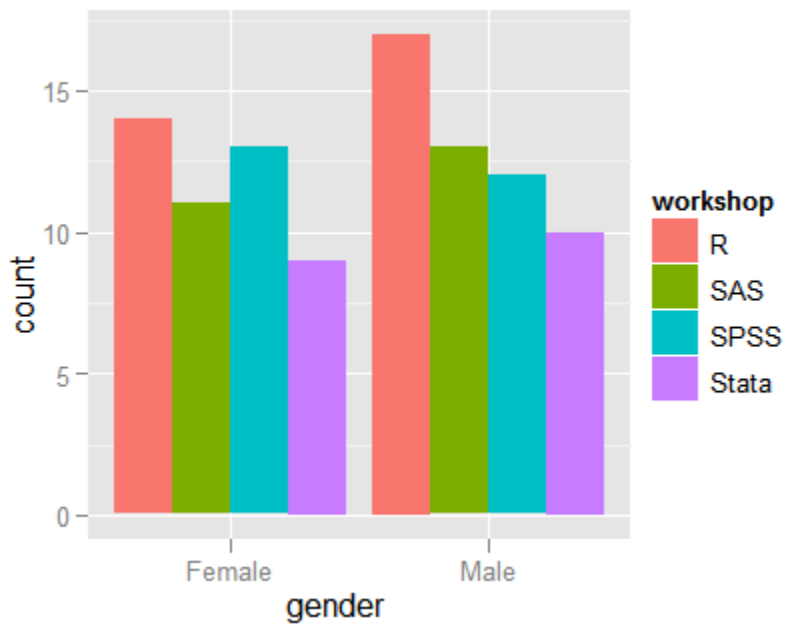
In the plot above, the height of the bars represents the total number of males and females. This is fine if you want to compare counts, but if you want to compare proportions of each gender that took each class, you would have to make the bars equal heights. You can do that by simply changing the position to “fill”.

```
1 ggplot(mydata100, aes(gender, fill=workshop) )  
2 +geom_bar(position="fill")
```



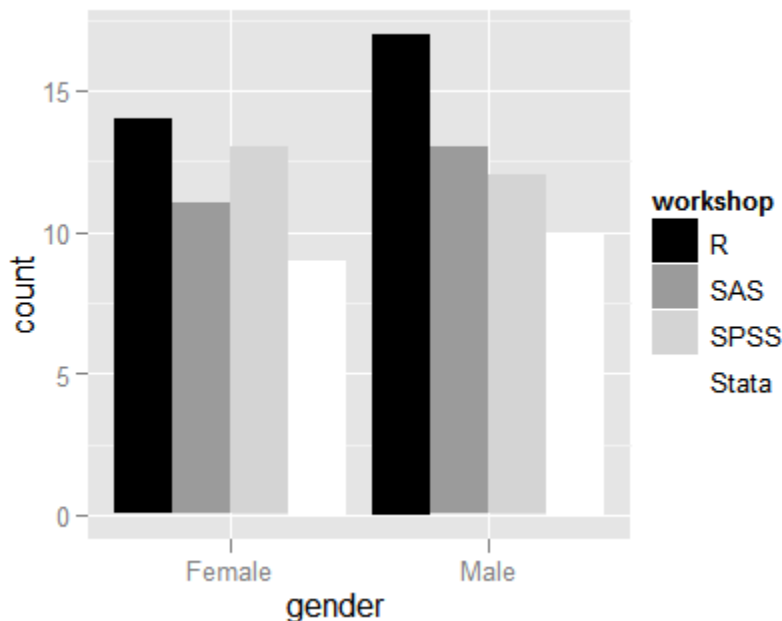

Here is the same plot changing only the bar position to be “dodge”.

```
ggplot(mydata100, aes(gender, fill=workshop ) )  
+ geom_bar(position="dodge")
```



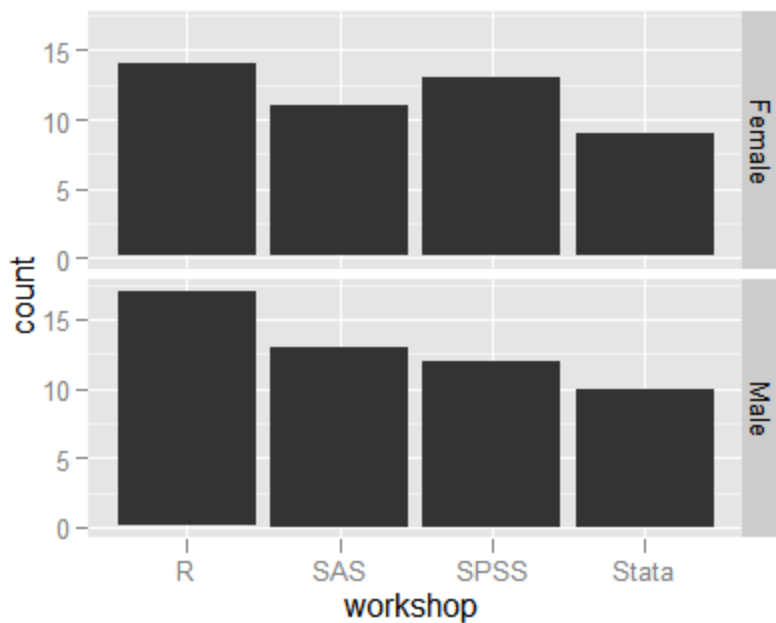
You can change any of the above colored graphs to shades of grey by simply adding the `scale_fill_grey()` function. Here is the plot immediately above repeated in greyscale.

```
ggplot(mydata100, aes(gender, fill=workshop ) )  
+ geom_bar(position="dodge") +scale_fill_grey(start  
= 0, end = 1)
```



You can get the same information that is in the above plot by making small separate plots for one of the groups. You can accomplish that with the `facet_grid()` function. It accepts a formula in the form “rows ~ columns”, so using “gender ~ .” asks for two rows for the genders (three if we had not removed missing values) and no columns.

```
ggplot(mydata100, aes(workshop) ) + geom_bar()  
+ facet_grid(gender ~ .)
```

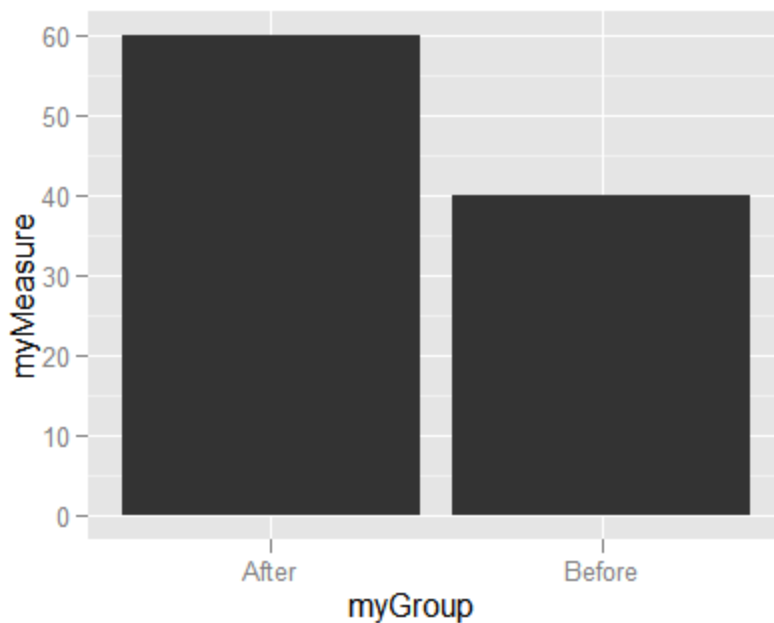


Pre-summarized Data

The ggplot2 package summarizes your data for you. If it is already summarized, you can create a small data frame of the results to plot.

```
myTemp <- data.frame(myGroup=factor( c("Before","After")  
,myMeasure=c(40, 60))
```

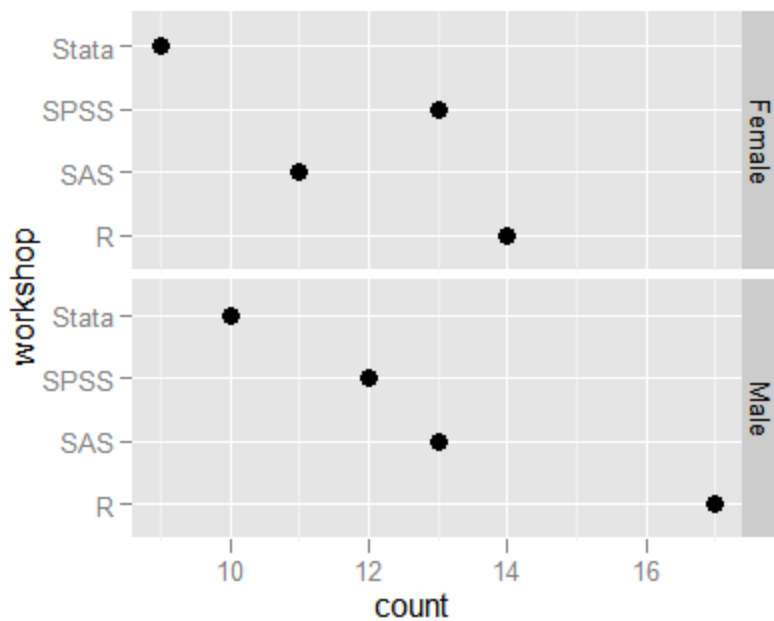
```
ggplot(data=myTemp, aes(myGroup, myMeasure) ) +  
geom_bar(stat = "identity")
```



Dot Charts

Dot charts are similar to bar charts, but since they are plotting points on both an x- and y-axis, they require a special variable called “..count..”. It calculates the counts and lets you plot them on the y-axis. The points use the “bin” statistic. Since dot charts are usually shown “sideways” I am adding the `coord_flip()` function.

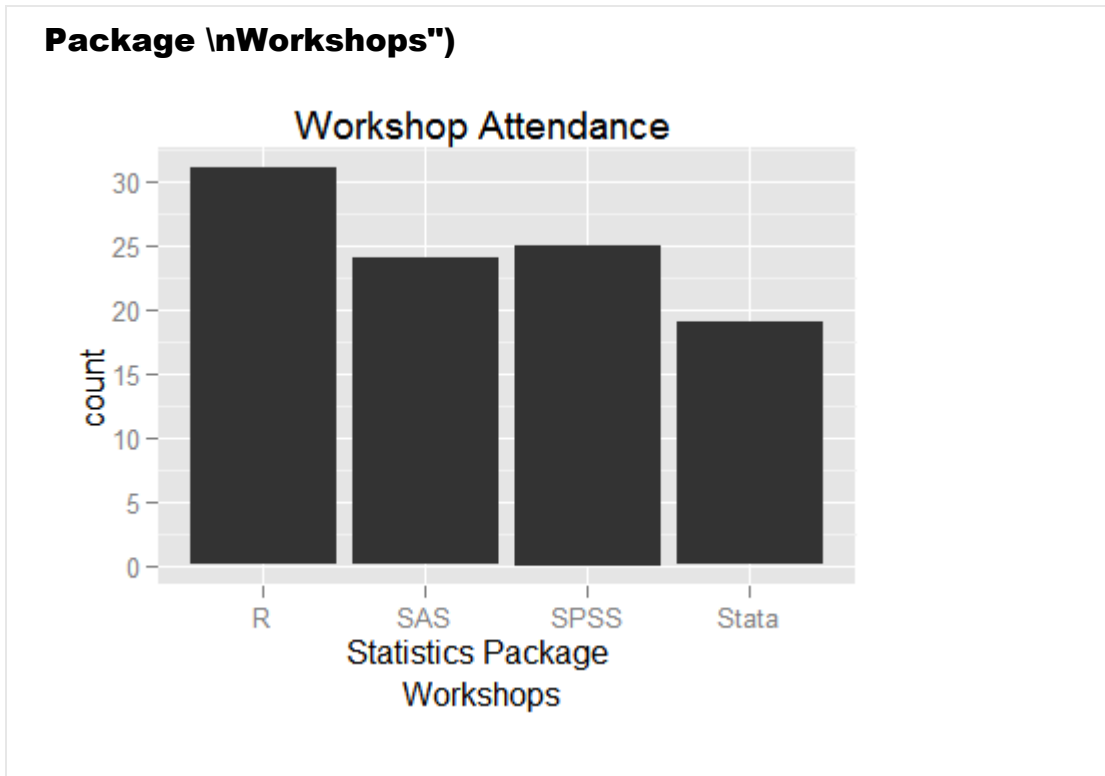
```
1 ggplot(mydata100,  
2   aes(workshop, ..count.. ) ) +  
3   geom_point(stat = "count", size = 4) +  
4   coord_flip()+  
5   facet_grid( gender ~ . )
```



Adding Titles and Labels

To add a title, use the `labs()` function and its `title` and `x` or `y` arguments. The character sequence “\n” tells R to go to a new line in all R packages.

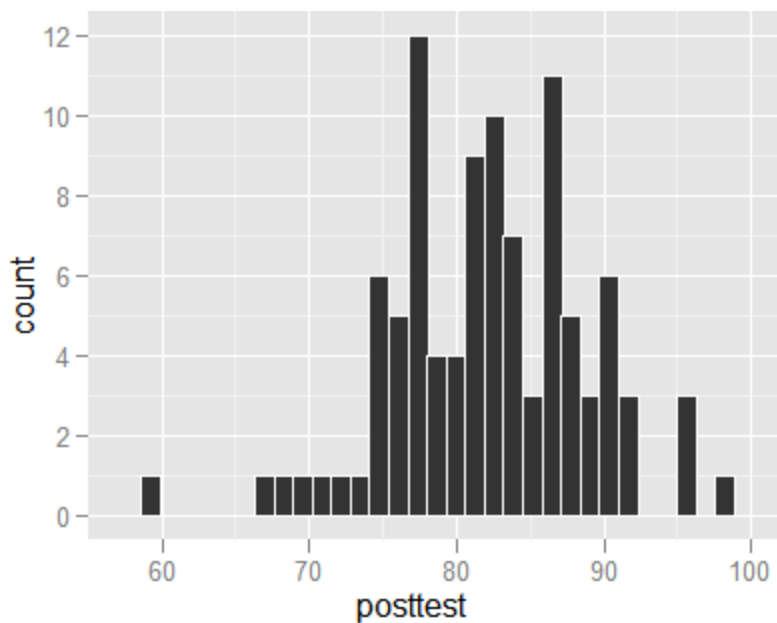
```
ggplot(mydata100, aes(workshop, ..count..)) + geom_bar()  
+ labs(title = "Workshop Attendance", x = "Statistics")
```



Histograms

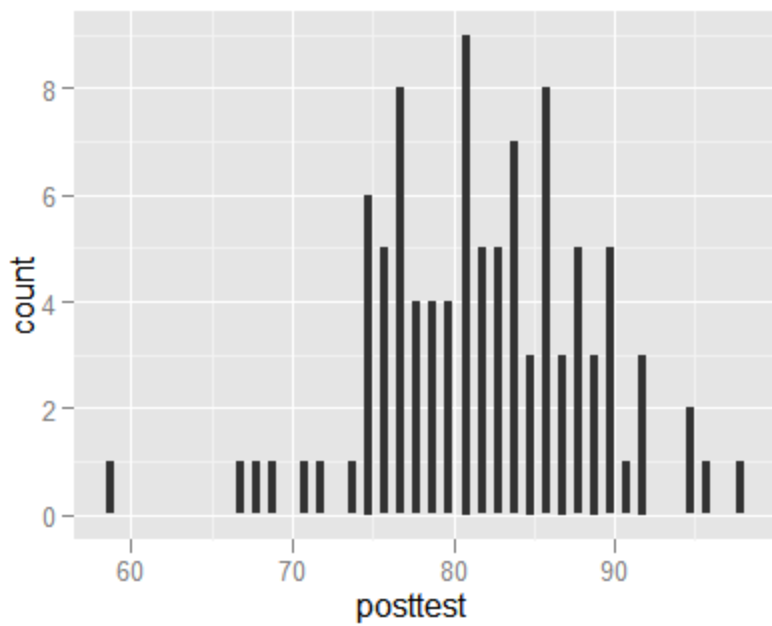
Recall from our first example that you can use `qplot` to get a quick histogram: `qplot(posttest)`. However, as things get more complicated, `ggplot()` is easier to control. The `geom_histogram` function is all you need. I have set the color of the bar edges to white. Without that, the bars all run together in the same shade of grey.

```
ggplot(mydata100, aes(posttest) )  
+ geom_histogram(color="white")
```



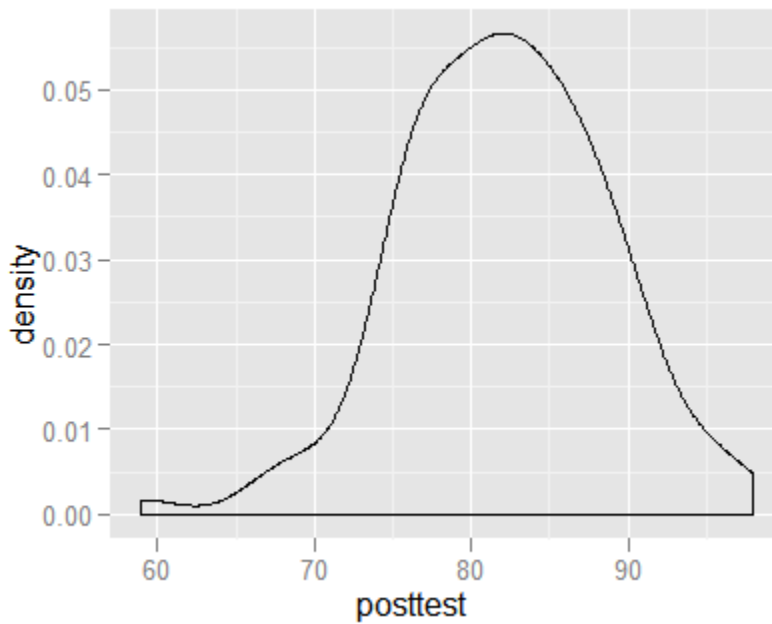
You can change the number of bars used using the `binwidth` argument. Since this many bars do not touch, I did not bother setting the edge color to white.

```
ggplot(mydata100, aes(posttest) ) +  
  geom_histogram(binwidth = 0.5)
```



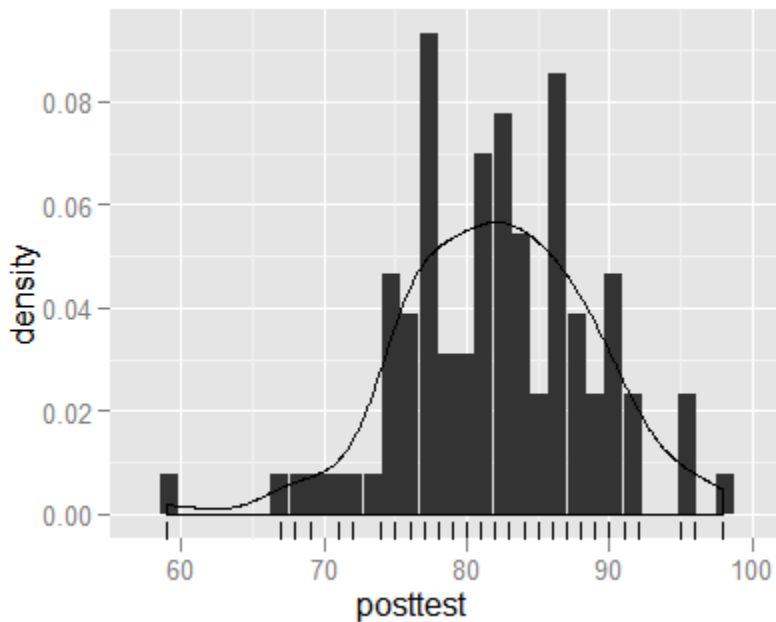
If you prefer a density plot, that is easy too.

```
ggplot(mydata100, aes(posttest)) +  
  geom_density()
```



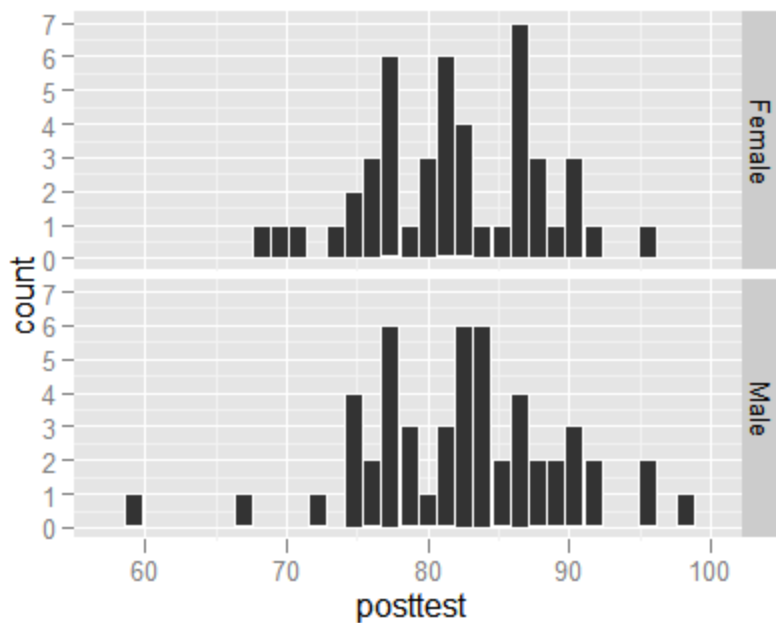
It is easy to layer many different geometric objects onto your plots. In this case to get the same axis on the histogram as the density uses, I used a special ggplot2 variable named “..density..” on the y-axis. I also added a “rug” of carpet-like tick marks on the x-axis using `geom_rug`.

```
ggplot(data=mydata100) + geom_histogram( aes(posttest,  
..density..) ) + geom_density( aes(posttest, ..density..) ) +  
geom_rug( aes(posttest) )
```



Comparing group histograms is easy when you facet them.

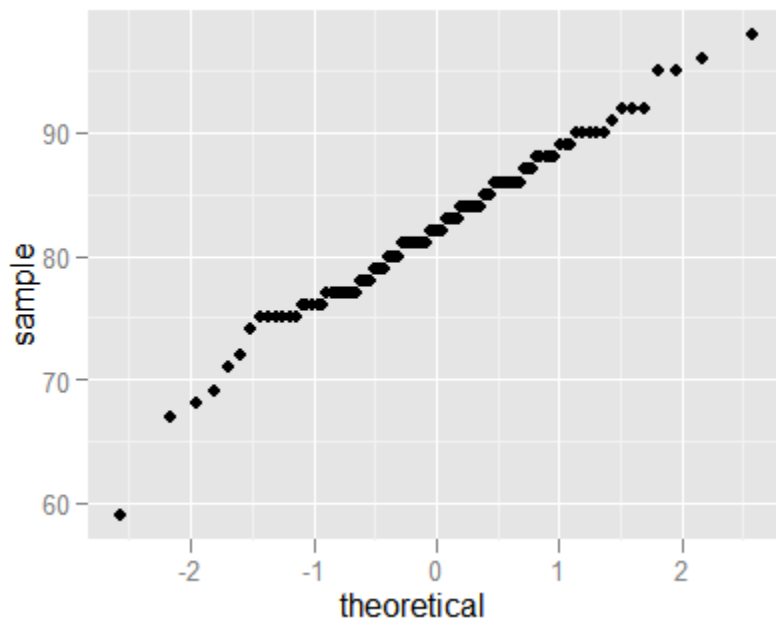
```
ggplot(mydata100, aes(posttest) ) +  
geom_histogram(color = "white") +  
facet_grid(gender ~ .)
```



Normal QQ Plots

Normal QQ plots are done in ggplot with the `stat_qq()` function and the `sample` aesthetic.

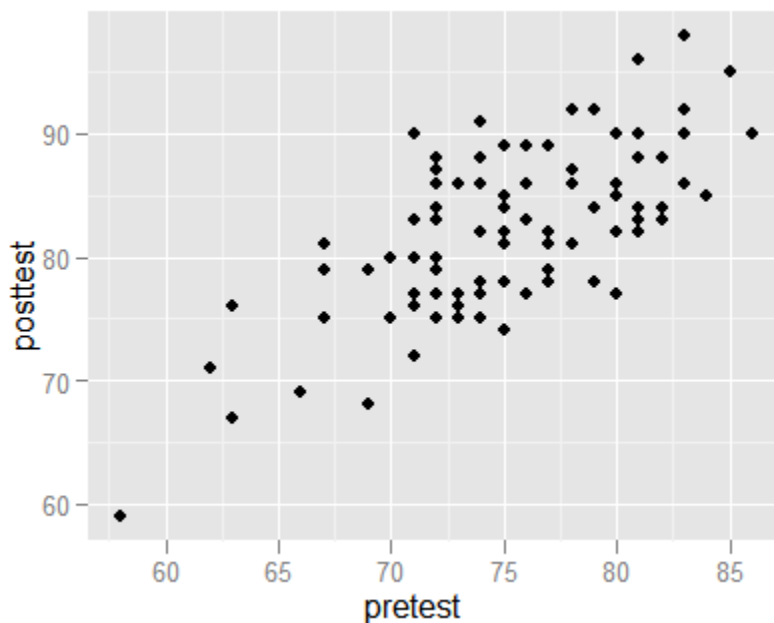
```
ggplot(mydata100, aes(sample = posttest) ) +  
  stat_qq()
```



Scatter and Line Plots

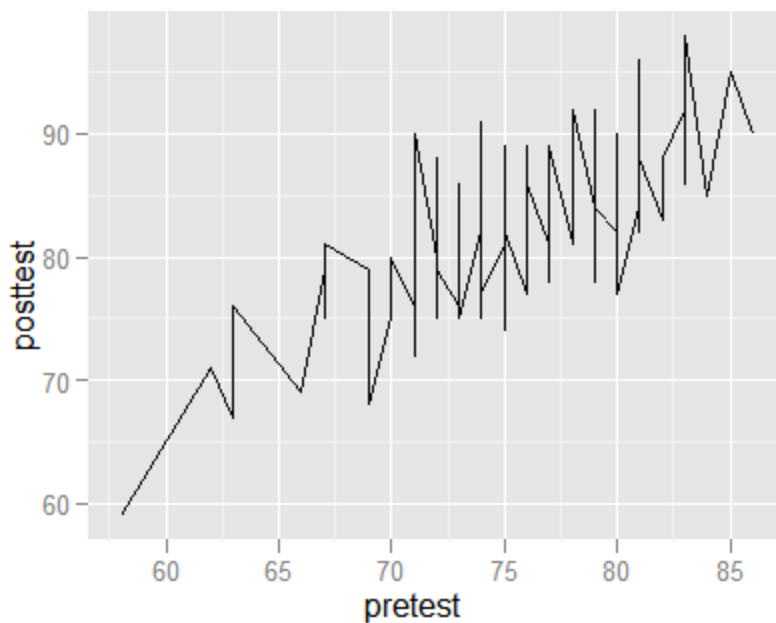
Various type of scatter and line plots can be done using different geoms as shown below. You can, of course, add multiple geoms to a plot. For example, you might want both points and lines, in which case you would simply add both geoms.

```
ggplot(mydata100, aes(pretest, posttest)) +  
  geom_point()
```



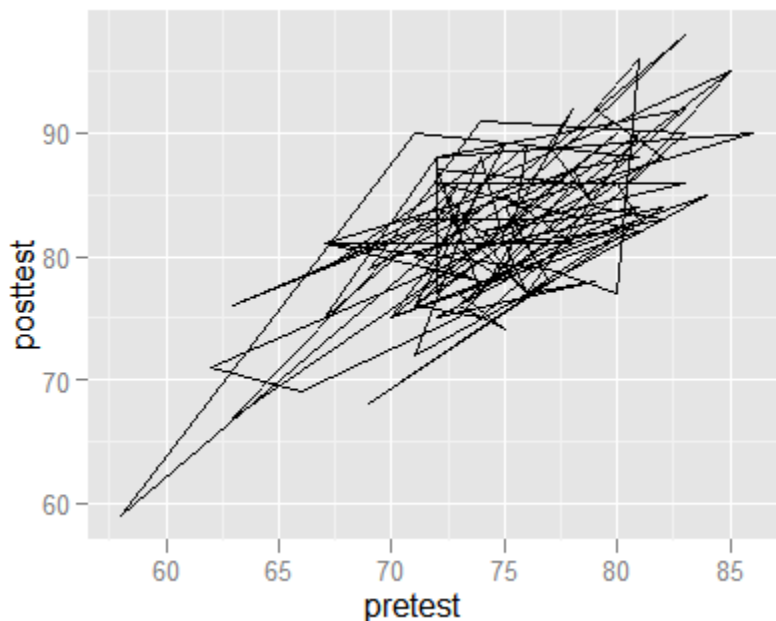
When you add a line geom, the ggplot sorts the data along the x-axis automatically. If you had time-series data that were not sorted by date, it would do so.

```
ggplot(mydata100, aes(pretest, posttest)) +  
  geom_line()
```



The path geom leaves the order of the data as it is; it does not sort it before connecting the points. This would make more sense if we had geographic mapping data. See the books for more examples.

```
ggplot(mydata100, aes(pretest, posttest)) +  
  geom_path()
```



Box Plots

The ggplot package offers considerable control over how you can do box plots. Here I plot the raw points and then the boxes on top of them. This hides the points that are actually in the middle 50% of the data. They are usually dense and of less interest than the points that are further out. If you have a lot of data you might consider using `geom_jitter()` to spread the points around, preventing over-plotting.

```
ggplot(mydata100, aes(workshop, posttest )) +  
geom_point() + geom_boxplot()
```

